Uninitialized algorithms for relocation

Document #:	P3516R1 [Latest] [Status]
Date:	2025-02-11
Project:	Programming Language C++
Audience:	LEWG

Reply-to:	Louis Dionne <ldionne.2@gmail.com> Giuseppe D'Angelo <giuseppe.dangelo@kdab.com>

Contents

- 1 Changelog
- 2 Introduction
- 3 Motivation
- 4 Proposal
- 5 Usage examples
  - 5.1 `vector::emplace` / `vector::insert`
  - 5.2 `vector::erase`
  - 5.3 `vector::emplace_back` / `vector::push_back`
- 6 Compatibility with relocation proposals
- 7 Wording
  - 7.1 [version.syn]
  - 7.2 [algorithms.requirements]
  - 7.3 [specialized.algorithms.general]
  - 7.4 [special.mem.concepts]
  - 7.5 [memory.syn]
  - 7.6 [specialized.algorithms]
- 8 Acknowledgements

§ 1 Changelog

- R1 (LEWG in Hagenberg)
  - Rebased on top of P2786R11, after the modifications approved by LEWG on 2025-01-14.
  - Wording fixes.
- R0 (Pre-Hagenberg)
  - First revision.

§ 2 Introduction

P2786 is currently making progress through EWG and LEWG. In Poland, LEWG expressed a strong desire for that language feature to come with a proper user-facing library interface. This paper proposes such an interface. That interface is heavily influenced by the uninitialized algorithms that were originally proposed in P1144 which, after implementation experience in libc++, seem to be almost exactly what we need.

This paper aims to provide the high-level algorithms that all programmers will want to use, while purposefully not touching on some of the more contentious design points of P2786/P1144. The high-level algorithms proposed in this paper are compatible

with essentially any definition of “trivial relocation” we might introduce in the language in C++26 or later.

3 Motivation

Today, many algorithms and containers use some flavor of assignment, either as an implementation detail or as part of their API promise. In cases where the use of assignment is merely an implementation detail, writing the code in terms of relocation can often result in cleaner code and improved performance (since move-construction is often faster than move-assignment).

Furthermore, in a future where the language itself provides a definition of relocation (and in particular trivial relocation), algorithms and containers written in terms of relocation today would benefit from a massive speedup for trivially relocatable types without requiring any change. Indeed, the algorithms introduced in this paper boil down to `memmove` (or `memcpy` if we can prove the absence of overlap) in trivial cases, which are fairly common.

4 Proposal

We propose adding algorithms `std::uninitialized_relocate`, `std::uninitialized_relocate_n`, and `std::uninitialized_relocate_backward`. We also add the equivalent `ranges::` functions and the parallel algorithm overloads. At a high level, these algorithms relocate a range of objects from one memory location to another, which is a very useful primitive when writing containers and algorithms.

However, the library interface proposed here is intentionally independent of the details of the underlying language feature.

Design points:

— These functions behave similarly to the other uninitialized algorithms like `std::uninitialized_copy`, with the difference that the new proposed relocation algorithms support overlapping input/output ranges like `std::move` and `std::move_backward`. That is necessary in order to support most use cases where one is relocating a range a bit “to the right” or a bit “to the left” of its current location.

— Unlike P2786 which talks of *trivial relocation*, these algorithms talk of *relocation* in the general sense. This is because it’s generally more useful (and easier!) to write an algorithm or data structure operation in terms of general relocation, even if it is not trivial (in which case it is move + destroy). Even non-trivial relocation is usually faster than the equivalent assignment-based algorithm. By encouraging users to write their code in terms of general relocation today, we are preparing the terrain for when the language gets a notion of trivial relocation, and in the meantime we let implementations optimize under the as-if rule.

— The algorithms provide an exception guarantee that in case of an exception, the whole source range is destroyed and the whole destination range as well. Otherwise, the ranges would contain an unknown number of alive objects, making it impossible for the caller to perform a clean up. This turns out to work well with what e.g. `std::vector::insert` requires in order to provide the basic exception safety guarantee (see the examples below).

— The algorithms accept `input_iterators` (or `bidirectional_iterators` for the backward variant) for the input range. This iterator category is sufficient to implement these algorithms properly. However, it does mean that the range overlap precondition cannot be checked at runtime for some iterator categories. In practice, we would expect an implementation that wishes to check this precondition to do so whenever the input range is at least random access, which should be the vast majority of use cases.

— The algorithms accept `forward_iterators` for the output range, which is necessary to perform cleanup if an exception is thrown.

— The proposed API facilities are intended to be as consistent as possible with existing uninitialized algorithms, with the few differences being a direct consequence of the fact that the source range does not contain any elements after performing the operation.

5 Usage examples

In these examples, we ignore allocator support for simplicity, and we use `iterator` instead of `const_iterator` as the parameter types to lighten the code. The “real” code for implementing these functions does not look meaningfully different.

5.1 `vector::emplace` / `vector::insert`

Assignment based

```
template <class ...Args>
constexpr iterator
vector<T>::emplace(iterator position, Args&&...
        args) {
  if [[unlikely]] (size() == capacity()) {
    // slow path where we grow the vector
  }

  if (position == end()) {
    std::construct_at(std::to_address(position),
                      std::forward<Args>
        (args)...);
    ++end_;
  } else {
    T tmp(std::forward<Args>(args)...);

    // Create a gap at the end by moving the
        last
    // element into the "new last position".
    std::construct_at(std::to_address(end()),
                      std::move(back()));
    ++end_;

    // The element before the last one is moved-
        from:
    // shift everything else by one position to
        the
    // right to open a gap at the insert
        location.
    std::move_backward(position, end() - 2,
        end() - 1);

    // Now there's a gap at the insert location,
        move the
    // new element into it.
    *position = std::move(tmp);
  }
  return position;
}
```

This paper

```
template <class ...Args>
constexpr iterator
vector<T>::emplace(iterator position, Args&&... args)
        {
  if [[unlikely]] (size() == capacity()) {
    // slow path where we grow the vector
  }

  if (position == end()) {
    std::construct_at(std::to_address(position),
                      std::forward<Args>(args)...);
    ++end_;
  } else {
    T tmp(std::forward<Args>(args)...);

    // Create a gap inside the vector by relocating
    // everything to the right.
    try {
      std::uninitialized_relocate_backward(position,
        end(),
                                           end() + 1);
    } catch (...) {
      // basic guarantee: vector lost its tail
      end_ = std::to_address(position);
      throw;
    }

    // Move-construct the new value into the gap
        created above.
    try {
      std::construct_at(std::to_address(position),
        std::move(tmp));
    } catch (...) {
      // basic guarantee: vector lost its tail
      std::destroy(position + 1, end() + 1);
      end_ = std::to_address(position);
      throw;
    }

    ++end_;
  }
  return position;
}
```

A few things are worth noting:

— The relocation-based implementation looks more complicated than the assignment-based one, but in reality the complexity only stems from exception handling. The algorithm is conceptually simpler because we just relocate things around instead of creating artificial gaps that contain moved-from objects, like in the assignment-based one.

— Both implementations provide the basic exception safety guarantee, but they provide it differently. After an exception, the assignment-based implementation may leave elements in the `[position, end())` range in a moved-from state, and the vector size may or may not have been incremented by 1. Whether this is actually the case or not depends on which operation exactly throws. Either way, the user has no direct means to know which elements have been left in a moved-from state. The relocation-based implementation instead ends up with the elements in `[position, end())` being erased from the vector. It's unclear whether one behavior is really superior over the other, but both are conforming.

— There is no mention of *trivial* relocation in the implementation. We relocate elements even when that is not equivalent to `std::memcpy` (or whatever the language definition would be).

— An implementation that considers the usage of move-assignment to be part of the function's API may want to preserve that behavior, in which case the relocating code path may be guarded by something like P2786's `is_replaceable_v<T>` (and otherwise fallback to move-assignment). In the absence of `is_replaceable`, a conservative implementation could use `std::is_trivially_copyable` as a guard.

§ 5.1.1 This code with P2786 only

Implementing a similar relocation-based `vector::emplace` with P2786's facilities only but without the facilities in this paper would produce the following code:

```
template <class ...Args>
constexpr iterator
vector<T>::emplace(iterator position, Args&&... args) {
  if [[unlikely]] (size() == capacity()) {
    // slow path where we grow the vector
  }

  auto relocate_via_assignment = [&] {
    T tmp(std::forward<Args>(args)...);

    // Create a gap at the end by moving the last
    // element into the "new last position".
    std::construct_at(std::to_address(end()),
                      std::move(back()));
    ++end_;

    // The element before the last one is moved-from:
    // shift everything else by one position to the
    // right to open a gap at the insert location.
    std::move_backward(position, end() - 2, end() - 1);

    // Now there's a gap at the insert location, move the
    // new element into it.
    *position = std::move(tmp);
  };

  if (position == end()) {
    std::construct_at(std::to_address(position),
                      std::forward<Args>(args)...);
    ++end_;
  } else {
    // Create a gap inside the vector by relocating
    // everything to the right.
    if constexpr (std::is_trivially_relocatable_v<T>) {
      if consteval {
        relocate_via_assignment();
      } else {
        union { T tmp; };
        std::construct_at(std::addressof(tmp), std::forward<Args>(args)...);

        std::trivially_relocate(std::to_address(position), std::to_address(end()),
                                std::to_address(end()) + 1);

        // Trivially relocate the new value into the gap created above.
        std::trivially_relocate(std::addressof(tmp), std::addressof(tmp) + 1,
                                std::to_address(position));
        ++end_;
      }
    } else {
      relocate_via_assignment();
    }
  }
  return position;
}
```

§ 5.2 `vector::erase`

Assignment based	This paper

`constexpr iterator vector<T>::erase(iterator first, iterator last) {`
`  if (first == last)`
`  return last;`
 	
`  auto new_end = std::move(last, end(), first);`
`  std::destroy(new_end, end());`
 	
`  end_ -= (last - first);`
`  return first;`
`}`
	`constexpr iterator vector<T>::erase(iterator first, iterator last) {`
	`  if (first == last)`
	`  return last;`
	
	`  // Destroy the range being erased and relocate the`
	`  // tail of the vector into the created gap.`
	`  std::destroy(first, last);`
	`  try {`
	`  std::uninitialized_relocate(last, end(), first);`
	`  } catch (...) {`
	`  end_ = std::to_address(first); // vector lost its tail`
	`  throw;`

Assignment based	This paper

	}
	
	end_ -= (last - first);
	return first;
	}

Things worth noting:

— Again, there is no mention of *trivial* relocation above, since we write the algorithm in term of general relocation.

— Like for `vector::emplace`, an implementation that considers the usage of move-assignment to be part of the function's API may want to preserve that behavior by further constraining the usage of relocation.

— Pedantically, the relocation-based implementation violates an exception constraint on `std::vector::erase`. The current specification requires `vector::erase` not to throw an exception unless `T`'s *move-assignment operator* throws. The implementation above can also throw if the *move-constructor* throws. We view this as an overspecification in the Standard that could be relaxed – this is probably an artifact of the fact that implementations were assumed to use assignment. To make the definition above pedantically correct, we should only use relocation when `is_nothrow_move_constructible<T>` is satisfied, and otherwise use assignment.

§ 5.3 `vector::emplace_back` / `vector::push_back`

Note that the vector growth policy presented here is more naive than what `std::vector` typically does. Also keep in mind that we must provide the strong exception safety guarantee here.

Move-construction based

```
template <class ...Args>
constexpr reference
vector<T>::emplace_back(Args&& ...args) {
  if [[likely]] (size() < capacity()) {
    std::construct_at(std::to_address(end()),
                      std::forward<Args>(args)...);
    ++end_;
    return back();
  }

  // Move or copy all the elements into a large
  // enough vector.
  vector<T> tmp;
  tmp.reserve((size() + 1) * 2);
  std::construct_at(tmp.begin_ + size(),
                    std::forward<Args>(args)...);
  // guard destruction in case of exception

  for (auto& element : *this) {
    [[assume(tmp.size() < tmp.capacity())]];
    tmp.emplace_back(std::move_if_noexcept(element));
  }

  // disengage guard
  ++tmp.end_;

  swap(tmp);
  return back();
}
```

This paper

```
template <class ...Args>
constexpr reference
vector<T>::emplace_back(Args&& ...args) {
  if [[likely]] (size() < capacity()) {
    std::construct_at(std::to_address(end()),
                      std::forward<Args>(args)...);
    ++end_;
    return back();
  }

  // Relocate all the elements into a large enough
       vector,
  // or copy if moving might throw.
  vector<T> tmp;
  tmp.reserve((size() + 1) * 2);
  std::construct_at(tmp.begin_ + size(),
                    std::forward<Args>(args)...);
  // guard destruction in case of exception

  if constexpr (is_nothrow_relocatable_v<T>) { // from
       P2786
    tmp.end_ = std::uninitialized_relocate(begin(),
       end(),
                                           tmp.begin_);
    end_ = begin_;
  } else {
    for (auto& element : *this) {
      [[assume(tmp.size() < tmp.capacity())]];
      tmp.emplace_back(std::move_if_noexcept(element));
    }
  }

  // disengage guard
  ++tmp.end_;

  swap(tmp);
  return back();
}
```

Things worth noting:

— Again, there is no mention of *trivial* relocation: we write the algorithm based on general relocation.

— The above implementation omits to support the case where `args...` is in fact a single object located inside the vector we're inserting in. Supporting that essentially requires creating a temporary value or constructing the new element before we start moving from the current vector, but does not meaningfully change the code.

— We use `std::move_if_noexcept` even in the case of a potentially throwing move constructor because we must handle non-copyable types with a potentially throwing move constructor (in which case the strong exception safety guarantee is waived).

§ 6 Compatibility with relocation proposals

The current version of the proposal is based on P2786 trivial relocation library APIs, as approved by LEWG during the 2025-01-14 meeting.

In general it's worth noting that our specification can be easily extended to a future notion of language relocation; the algorithms themselves won't need to change; only the proposed exposition-only entities will need centralized fixes. We consider this a major feature of the proposed design.

§ 7 Wording

Note: the following wording assumes that P2786R11 (as approved by LEWG, see above) has already been merged.

§ 7.1 [version.syn]

Modify the feature-testing macro for `__cpp_lib_raw_memory_algorithms` to match the date of adoption of the present proposal:

```
-#define __cpp_lib_raw_memory_algorithms             202411L // also in <memory>
+#define __cpp_lib_raw_memory_algorithms             ??????L // also in <memory>
```

§ 7.2 [algorithms.requirements]

Modify 26.2 [algorithms.requirements] as shown:

(4.?) — If an algorithm's template parameter is named `InputIterator`, `InputIterator1`, or `InputIterator2`, the template argument shall meet the *Cpp17InputIterator* requirements ([input.iterators]).

(4.?) — If an algorithm's template parameter is named `OutputIterator`, `OutputIterator1`, or `OutputIterator2`, the template argument shall meet the *Cpp17OutputIterator* requirements ([output.iterators]).

(4.?) — If an algorithm's template parameter is named `ForwardIterator`, `ForwardIterator1`, `ForwardIterator2`, ~~or~~ `NoThrowForwardIterator`, `NoThrowForwardIterator1, or NoThrowForwardIterator2,` the template argument shall meet the *Cpp17ForwardIterator* requirements ([forward.iterators]) if it is required to be a mutable iterator, or model `forward_iterator` ([iterator.concept.forward]) otherwise.

(4.?) — ~~If an algorithm's template parameter is named `NoThrowForwardIterator`, the template argument is also required to have the property that no exceptions are thrown from increment, assignment, or comparison of, or indirection through, valid iterators.~~

(4.?) — If an algorithm's template parameter is named `BidirectionalIterator`, `BidirectionalIterator1`, ~~or~~ `BidirectionalIterator2`, `NoThrowBidirectionalIterator1, or NoThrowBidirectionalIterator2`, the template argument shall meet the *Cpp17BidirectionalIterator* requirements ([bidirectional.iterators]) if it is required to be a mutable iterator, or model `bidirectional_iterator` ([iterator.concept.bidir]) otherwise.

(4.?) — If an algorithm's template parameter is named `RandomAccessIterator`, `RandomAccessIterator1`, or `RandomAccessIterator2`, the template argument shall meet the *Cpp17RandomAccessIterator* requirements ([random.access.iterators]) if it is required to be a mutable iterator, or model `random_access_iterator` ([iterator.concept.random.access]) otherwise.

(4.?) — If an algorithm's template parameter is named `NoThrowForwardIterator`, `NoThrowForwardIterator1`, `NoThrowForwardIterator2`, `NoThrowBidirectionalIterator1`, or `NoThrowBidirectionalIterator2`, the template argument is also required to have the property that no exceptions are thrown from increment, assignment, or comparison of, or indirection through, valid iterators.

§ 7.3 [specialized.algorithms.general]

Modify 26.11.1 [specialized.algorithms.general] as shown:

2 — Unless otherwise specified, if an exception is thrown in the following algorithms, objects constructed by a placement *new-expression* (7.6.2.8 [expr.new]) or whose lifetime has started due to a call to `trivially_relocate` are destroyed in an unspecified order before allowing the exception to propagate.

…

4 — Some algorithms specified in [specialized.algorithms] make use of the ~~exposition-only function template *voidify*~~ following exposition-only entities:

```
template<class T>
  constexpr void* voidify(T& obj) noexcept {
    return addressof(obj);
  }

+template<class Source, class Dest>
+ concept relocatable-from =
+   same_as<Source, Dest> &&
+   (move_constructible<Dest> || is_trivially_relocatable_v<Dest>);
+
+template<class T>
+ requires move_constructible<T>
+  constexpr T* relocate-via-move-and-destroy(T* dest, T* source) {
+    struct guard { T *t; ~guard() { destroy_at(t); } } g(source);
+    return ::new (voidify(*dest)) T(std::move(*source));
+  }
+
+template<class T>
+ requires relocatable-from<T, T>
+  constexpr T* relocate-at(T* dest, T* source)
+    noexcept(is_nothrow_relocatable_v<T>)
+  {
+    if constexpr (is_trivially_relocatable_v<T> && is_move_constructible_v<T>) {
+      if consteval {
+        return relocate-via-move-and-destroy(dest, source);
+      } else {
+        return trivially_relocate(source, source + 1, dest);
+      }
+    } else if constexpr (is_trivially_relocatable_v<T>) {
+      return trivially_relocate(source, source + 1, dest);
+    } else {
+      return relocate-via-move-and-destroy(dest, source);
+    }
+  }
```

---

§ 7.4 [special.mem.concepts]

Add at the end of 26.11.2 [special.mem.concepts]:

```
template<class I>
concept nothrow-bidirectional-iterator = // exposition only
  nothrow-forward-iterator<I> &&
  bidirectional_iterator<I> &&
  nothrow-sentinel-for<I, I>;
```

7 — [*Note*: This concept allows some `bidirectional_iterator` ([iterator.concept.bidir]) operations to throw exceptions. — *end note*]

```
template<class R>
concept nothrow-bidirectional-range = // exposition only
  nothrow-forward-range<R> &&
  nothrow-bidirectional-iterator<iterator_t<R>>;
```

§ 7.5 [memory.syn]

In 20.2.2 [memory.syn], add to the `<memory>` header synopsis (before `[unique.ptr]`):

```
template<class NoThrowForwardIterator1, class NoThrowForwardIterator2>
  constexpr NoThrowForwardIterator2
    uninitialized_relocate(NoThrowForwardIterator1 first,    // freestanding
                           NoThrowForwardIterator1 last,
                           NoThrowForwardIterator2 result);
```

```
template<class ExecutionPolicy, class NoThrowForwardIterator1, class NoThrowForwardIterator2>
  constexpr NoThrowForwardIterator2
    uninitialized_relocate(ExecutionPolicy&& exec,    // see [algorithms.parallel.overloads]
                           NoThrowForwardIterator1 first,
                           NoThrowForwardIterator1 last,
                           NoThrowForwardIterator2 result);

template<class NoThrowForwardIterator1, class Size, class NoThrowForwardIterator2>
  constexpr pair<NoThrowForwardIterator1, NoThrowForwardIterator2>
    uninitialized_relocate_n(NoThrowForwardIterator1 first,    // freestanding
                             Size n,
                             NoThrowForwardIterator2 result);

template<class ExecutionPolicy, class NoThrowForwardIterator1, class Size, class NoThrowForwardIterator2>
  constexpr pair<NoThrowForwardIterator1, NoThrowForwardIterator2>
    uninitialized_relocate_n(ExecutionPolicy&& exec,    // see [algorithms.parallel.overloads]
                             NoThrowForwardIterator1 first,
                             Size n,
                             NoThrowForwardIterator2 result);

template<class NoThrowBidirectionalIterator1, class NoThrowBidirectionalIterator2>
  constexpr NoThrowBidirectionalIterator2
    uninitialized_relocate_backward(NoThrowBidirectionalIterator1 first,    // freestanding
                                    NoThrowBidirectionalIterator1 last,
                                    NoThrowBidirectionalIterator2 result);

template<class ExecutionPolicy, class NoThrowBidirectionalIterator1, class NoThrowBidirectionalIterator2>
  constexpr NoThrowBidirectionalIterator2
    uninitialized_relocate_backward(ExecutionPolicy&& exec, // see [algorithms.parallel.overloads]
                                    NoThrowBidirectionalIterator1 first,
                                    NoThrowBidirectionalIterator1 last,
                                    NoThrowBidirectionalIterator2 result);

namespace ranges {
  template<class I, class O>
    using uninitialized_relocate_result = in_out_result<I, O>;    // freestanding

  template<nothrow-forward-iterator I, nothrow-sentinel-for<I> S1,
           nothrow-forward-iterator O, nothrow-sentinel-for<O> S2>
    requires relocatable-from<iter_value_t<I>, iter_value_t<O>>
      constexpr uninitialized_relocate_result<I, O>
        uninitialized_relocate(I ifirst, S1 ilast, O ofirst, S2 olast);    // freestanding

  template<nothrow-forward-range IR, nothrow-forward_range<I> OR>,
    requires relocatable-from<range_value_t<IR>, range_value_t<OR>>
      constexpr uninitialized_relocate_result<borrowed_iterator_t<IR>, borrowed_iterator_t<OR>>
        uninitialized_relocate(IR&& in_range, OR&& out_range);    // freestanding

  template<class I, class O>
    using uninitialized_relocate_n_result = in_out_result<I, O>;    // freestanding

  template<nothrow-forward-iterator I,
           nothrow-forward-iterator O, nothrow-sentinel-for<O> S>
    requires relocatable-from<iter_value_t<I>, iter_value_t<O>>
      constexpr uninitialized_relocate_n_result<I, O>
        uninitialized_relocate_n(I ifirst, iter_difference_t<I> n, O ofirst, S olast);    // freestanding

  template<class I, class O>
    using uninitialized_relocate_backward_result = in_out_result<I, O>;    // freestanding

  template<nothrow-bidirectional-iterator I, nothrow-sentinel-for<I> S1,
           nothrow-bidirectional-iterator O, nothrow-sentinel-for<O> S2>
    requires relocatable-from<iter_value_t<I>, iter_value_t<O>>
      constexpr uninitialized_relocate_backward_result<I, O>
        uninitialized_relocate_backward(I ifirst, S1 ilast, O ofirst, S2 olast);    // freestanding

  template<nothrow-bidirectional-range IR, nothrow-bidirectional-range OR>
    requires relocatable-from<range_value_t<IR>, range_value_t<OR>>
      constexpr uninitialized_relocate_backward_result<borrowed_iterator_t<IR>, borrowed_iterator_t<OR>>
        uninitialized_relocate_backward(IR&& in_range, OR&& out_range);    // freestanding
}
```

§ 7.6 [specialized.algorithms]

7.6.1 [uninitialized.relocate]

Append to 26.11 [specialized.algorithms] a new subclause, `uninitialized_relocate` [uninitialized.relocate], with these contents:

```
template<class NoThrowForwardIterator1, class NoThrowForwardIterator2>
  constexpr NoThrowForwardIterator2
    uninitialized_relocate(NoThrowForwardIterator1 first,
                           NoThrowForwardIterator1 last,
                           NoThrowForwardIterator2 result);
```

? — *Preconditions*: `result` is not in the range `[first, last)`.

? — *Effects*: Equivalent to:

```
NoThrowForwardIterator2 first_result = result;
try {
  while (first != last) {
    relocate-at(addressof(*result), addressof(*first));
    ++first;
    ++result;
  }
} catch (...) {
  destroy(++first, last);
  destroy(first_result, result);
  throw;
}

return result;
```

```
namespace ranges {
  template<nothrow-input-iterator I, nothrow-sentinel-for<I> S1,
           nothrow-forward-iterator O, nothrow-sentinel-for<O> S2>
    requires relocatable-from<iter_value_t<I>, iter_value_t<O>>
      constexpr uninitialized_relocate_result<I, O>
        uninitialized_relocate(I ifirst, S1 ilast, O ofirst, O2 olast);

  template<nothrow-input-range IR, nothrow-forward_range<I> OR>,
    requires relocatable-from<range_value_t<IR>, range_value_t<OR>>
      constexpr uninitialized_relocate_result<borrowed_iterator_t<IR>, borrowed_iterator_t<OR>>
        uninitialized_relocate(IR&& in_range, OR&& out_range);
}
```

? — *Preconditions*: `ofirst` is not in the range `[ifirst, ilast)`.

? — *Effects*: Equivalent to:

```
O first_result = ofirst;
try {
  while (ifirst != ilast && ofirst != olast) {
    relocate-at(addressof(*ofirst), addressof(*ifirst));
    ++ifirst;
    ++ofirst;
  }
} catch (...) {
  destroy(first_result, ofirst);
  ++ifirst;
  while (ifirst != ilast && ofirst != olast) {
    destroy_at(addressof(*ifirst));
    ++ifirst;
    ++ofirst;
  }
  throw;
}

return {std::move(ifirst), ofirst};
```

7.6.2 [uninitialized.relocate_n]

Append to 26.11 [specialized.algorithms] a new subclause, `uninitialized_relocate_n` [uninitialized.relocate_n], with these contents:

```
template<class NoThrowForwardIterator1, class Size, class NoThrowForwardIterator2>
  constexpr pair<NoThrowForwardIterator1, NoThrowForwardIterator2>
```

```
  uninitialized_relocate_n(NoThrowForwardIterator1 first,
                           Size n,
                           NoThrowForwardIterator2 result);
```

? — *Preconditions*: `result` is not in the range `[first, first + n)`.

? — *Effects*: Equivalent to:

```
NoThrowForwardIterator2 first_result = result;
try {
  while (n > 0) {
    relocate-at(addressof(*result), addressof(*first));
    ++first;
    ++result;
    --n;
  }
} catch (...) {
  destroy_n(++first, --n);
  destroy(first_result, result);
  throw;
}

return {first, result};
```

```
namespace ranges {
  template<nothrow-forward-iterator I,
           nothrow-forward-iterator O, nothrow-sentinel-for<O> S>
    requires relocatable-from<iter_value_t<I>, iter_value_t<O>>
      constexpr uninitialized_relocate_n_result<I, O>
        uninitialized_relocate_n(I ifirst, iter_difference_t<I> n, O ofirst, S olast);
}
```

? — *Preconditions*: `ofirst` is not in the range `[ifirst, ifirst + n)`.

? — *Effects*: Equivalent to:

```
O first_result = ofirst;
try {
  while (n > 0 && ofirst != olast) {
    relocate-at(addressof(*ofirst), addressof(*ifirst));
    ++ifirst;
    ++ofirst;
    --n;
  }
} catch (...) {
  destroy(first_result, ofirst);
  ++ifirst;
  --n;
  while (n > 0 && ofirst != olast) {
    destroy_at(addressof(*ifirst));
    ++ifirst;
    ++ofirst;
    --n;
  }
  throw;
}

return {std::move(ifirst), ofirst};
```

§ 7.6.3 [uninitialized.relocate_backward]

Append to 26.11 [specialized.algorithms] a new subclause, `uninitialized_relocate_backward` [uninitialized.relocate_backward], with these contents:

```
template<class NoThrowBidirectionalIterator1, class NoThrowBidirectionalIterator2>
  constexpr NoThrowBidirectionalIterator2
    uninitialized_relocate_backward(NoThrowBidirectionalIterator1 first,
                                    NoThrowBidirectionalIterator1 last,
                                    NoThrowBidirectionalIterator2 result);
```

? — *Preconditions*: `result` is not in the range `(first, last]`.

? — *Effects*: Equivalent to:

```
NoThrowBidirectionalIterator2 result_last = result;
try {
  while (last != first) {
    --last;
    --result;
    relocate-at(addressof(*result), addressof(*last));
  }
} catch (...) {
  destroy(first, last);
  destroy(++result, result_last);
  throw;
}

return {last, result};
```

```
namespace ranges {
  template<nothrow-bidirectional-iterator I, nothrow-sentinel-for<I> S1,
           nothrow-bidirectional-iterator O, nothrow-sentinel-for<O> S2>
    requires relocatable-from<iter_value_t<I>, iter_value_t<O>>
      constexpr uninitialized_relocate_backward_result<I, O>
        uninitialized_relocate_backward(I ifirst, S1 ilast, O ofirst, S2 olast);

  template<nothrow-bidirectional-range IR, nothrow-bidirectional-range OR>
    requires relocatable-from<range_value_t<IR>, range_value_t<OR>>
      constexpr uninitialized_relocate_backward_result<borrowed_iterator_t<IR>, borrowed_iterator_t<OR>>
        uninitialized_relocate_backward(IR&& in_range, OR&& out_range);
}
```

? — *Preconditions*: `olast` is not in the range `(ifirst, ilast]`.

? — *Effects*: Equivalent to:

```
O result_last = olast;
try {
  while (ilast != ifirst && olast != ofirst) {
    --ilast;
    --olast;
    relocate-at(addressof(*olast), addressof(*ilast));
  }
} catch (...) {
  destroy(++olast, result_last);
  --ilast;
  while (ilast != ifirst && olast != ofirst) {
    --ilast;
    --olast;
    destroy_at(addressof(*ilast));
  }
  throw;
}

return {ilast, olast};
```

§ 8 Acknowledgements

Thanks to the authors of P2786 and P1144 for pushing for trivial relocation, for the numerous discussions that resulted in this paper, and for many bits of wording that we got inspiration from for this paper.